Multi-Agent Systems

HW6: Final Homework Assignment

MSc AI, VU

E.J. Pauwels

Version: December 1, 2025— **Deadline: Monday, December 29, 2025 (23h59)**

IMPORTANT

- This project is an **individual assignment.** So everyone should hand in their own copy.
- The questions below require programming. In addition to the report (addressing the questions and discussing the results of your experiments), also provide the code on which your results are based (preferably as Python notebooks). However, make sure that the **report is self-contained**, i.e. that it can be read and understood without the need to refer to the code. **Only the report will be graded, so it's NOT sufficient to submit only a notebook**.
- Store the pdf-report and the code in a zipped folder that you upload to canvas. **Use your name and VU ID as name for the zipped folder**, e.g.

  `Jane_Doe_1234567.zip`

- This assignment will be **graded.** The max score is 4 and will count towards your final grade.
- Your **final grade (on 10)** will be computed as follows:

  Assignments 1 thru 5 (max 1) + Individual assignment (max 4) + Final exam (max 5)

- If you fail to submit this assignment before the deadline (29 Dec), you get the opportunity to aim for the second deadline **Sunday, Jan 11, 2026 (23h59)**. However, in that case your score on 4 will be converted to a score on 3 (i.e. multiplied by 3/4), resulting in a max score of 3 out of 4.

1 Monte Carlo Tree Search (MCTS) (40%)

Searching a binary tree Consider a search space that is constructed as a *binary tree* of depth $d = 20$ (or more – if you're feeling lucky). Since the tree is binary, there are two *branches* (aka. edges, directions, decisions, etc) emanating from each node, each branch (call them L(eft) and R(ight)) pointing to a unique child node (except, of course, for the leaf nodes – see Fig 1). We can therefore assign to each node a unique "address" (A) that captures the route down the tree to reach that node (e.g. $A = LLRL$ – for an example, again see Fig 1). Finally, pick a random leaf-node as your target node and let's denote its address as A_t.

Assign values to leaf-nodes Next, assign to each of the 2^d leaf-nodes a value as follows:

- For every leaf-node i compute the **edit-distance** between its address A_i and the address A_t of the target leaf, i.e. $d_i := d(A_i, A_t)$.
- Recall that the **edit-distance** counts the number of positions at which two strings differ, e.g.:

$$d(LRLR, LRRR) = 1, \qquad d(LRRL, LLLL) = 2, \qquad d(RRLL, RLRR) = 3$$

- Finally, define the value x_i of leaf-node i to be a decreasing function of the distance $d_i = d(A_i, A_t)$ to the target node, and sprinkle a bit of noise for good measure: e.g.

$$x_i = Be^{-d_i/\tau} + \varepsilon_i$$

where B and τ are chosen such that most nodes have a non-negligible value (e.g. $B = 10$ and $\tau = d_{max}/5$ and $\varepsilon_i \sim N(0, 1)$). Just to be clear, the random noise ϵ_i is generated once and added to the value x_i which is then fixed. It is not regenerated for every rollout.

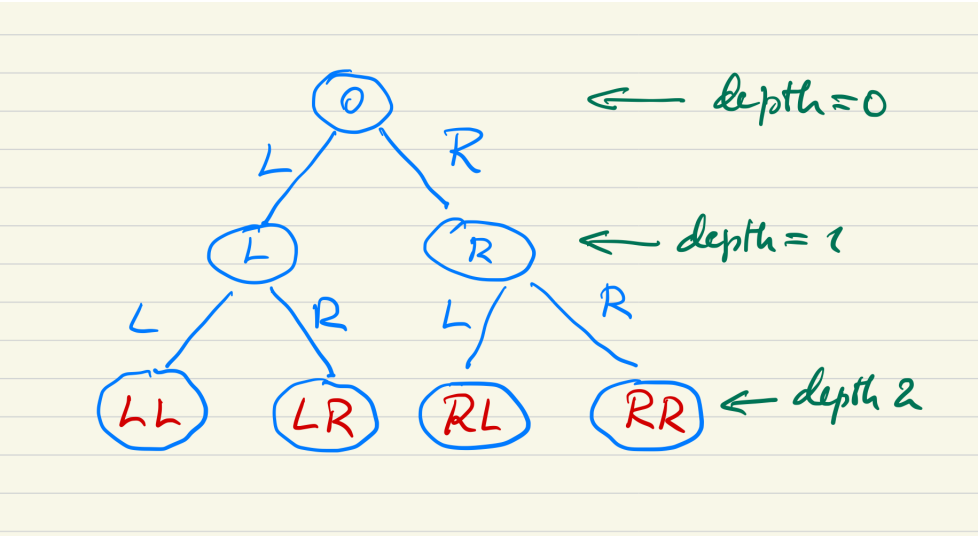

Figure 1: Example of binary tree of depth 2. The "address" of the leaf-nodes (layer 2) is indicated in red.

Questions

1. Implement the MCTS algorithm and apply it to the above tree to search for the optimal (i.e. highest) value.
2. Collect statistics on the performance and discuss the role of the hyperparameter c in the UCB-score. Some of the statistics you might want to consider to look at, include (but need not be restricted to):
    - Efficiency: Fraction of tree nodes visited;
    - Effectiveness: How close is the final answer to the actual optimal (e.g. as a percentage/fraction of the optimum)

Assume that the number MCTS-iterations starting in a specific root node is limited (e.g. to 10 or 50). Make a similar assumption for the number of roll-outs starting in a particular ("snowcap") leaf node (e.g. 1 or 5).

2 Markov Decision Process (MDP, 60%)

Assume that we have a circular MDP state-space that resembles the face of an analogue clock (see Fig. 2):

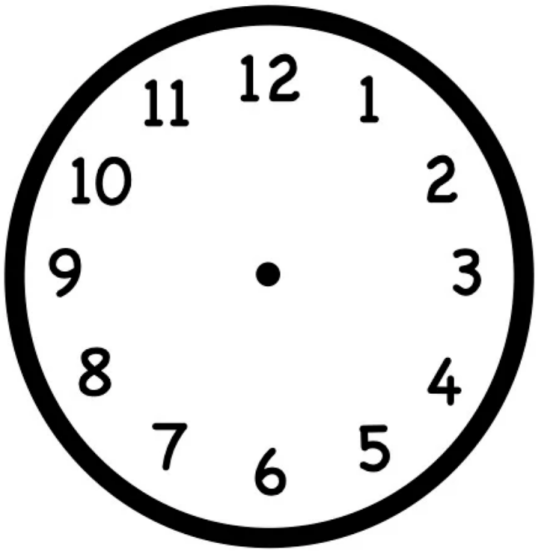

Figure 2: State space with 12 states organised as clock-face. State 12 (a.k.a. state 0) is an absorbing state.

The corresponding MDP is characterised as follows (see also Fig.2):

- **States** The state space consists of 12 states arranged on a circle. State 12 is an absorbing (goal) state with no outgoing transitions, while the remaining 11 states are regular states. For the sake of notational convenience, we will refer to the absorbing state both as $s = 0$ and $s = 12$.

- **Actions** In each regular state, the agent can choose between two actions: move clockwise to the next state (denoted R), or move counter-clockwise to the previous state (denoted L).
- **Transition probabilities** In states 5 through 11, transitions are deterministic: the chosen action is executed with certainty. In states 1, 2, 3 and 4, however, the probability that the chosen action is executed as intended equals $p_S = 1 - \varepsilon$ (where $0 \leq \varepsilon < 1/2$). As a consequence, with probability $p_F = 1 - p_S = \varepsilon$, the opposite action is executed instead.
- **Rewards** Any transition into a non-absorbing state (states 1–11) yields an immediate reward of $r_{NT} = -1$. A transition into the absorbing goal state yields an immediate reward of $r_T = 10$.
- **Discount factor** Unless stated otherwise, we assume $\gamma = 1$.

Optimal policy

Recall the Bellman optimality conditions for the value functions:

$$v^*(s) = \max_a q^*(s,a) \tag{1}$$

$$q^*(s,a) = \sum_{s'} p(s' \mid s,a) \left[r(s,a,s') + \gamma v^*(s') \right] \tag{2}$$

which can be combined in the following recursive equation for v^*:

$$v^*(s) = \max_a \sum_{s'} p(s' \mid s,a) \left[r(s,a,s') + \gamma v^*(s') \right] \tag{3}$$

Once v^* is known, q^* can be computed by a one-step lookup in eq. (2).

Questions

1. Construct for both actions ($a = L, R$):
   - The transition matrix T_a where $T_a(s,s') = p(s' \mid s,a)$;
   - The expected immediate reward vector $\mathbf{r}_a$ where

     $$\mathbf{r}_a(s) = \sum_{s'} p(s' \mid s,a) r(s,a,s')$$

   In terms of these quantities eq. (3) can be recast as a vector equation:

   $$\mathbf{v}^* = \max_a \left(\mathbf{r}_a + T_a \mathbf{v}^* \right) \tag{4}$$

2. **Assume $\varepsilon = 0$, i.e. all transitions are deterministic** In that case, the optimal policy π^* for this MDP, and the corresponding $\mathbf{v}^*$, are obvious: *Take the shortest path to the absorbing state $s = 0 = 12$.* Use eq. (4) to confirm this educated guess.
3. **Determine the optimal policy for noise-levels $\varepsilon = 0.2$ and $\varepsilon = 0.45$.** How are they different, explain why.

4. **Fit a parametrised policy** Consider the parametrised policy:

$$\pi_\theta(a = R \,|\, s) = \sigma(s - \theta)$$

where σ is the standard sigmoid:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Let $J(\theta)$ be the expected (total) return of paths τ generated under the policy π_θ, i.e.

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [G(\tau)]$$

where $G(\tau)$ is the sum of all the immediate rewards along the path τ. The starting point of each path is chosen at random (uniformly) and each path terminates in the absorbing state. The value θ^* that maximises J will depend on the noise level ε. Use Monte Carlo simulations to determine θ^* for the following two cases: (i) $\varepsilon = 0$ and (ii) $\varepsilon = 0.45$. How is this related to the previous results?